

AI Security Roadmap – Phase 1 Learning Report

Foundations: AI, ML, and GenAI Basics

Duration: Week 1

Goal: Be able to explain how a GenAI application works

Status: ☒ Complete (Theory + Hands-on Project)

Table of Contents

1. The Foundational Hierarchy (AI → ML → GenAI)
 2. Machine Learning Deep Dive
 3. Generative AI Deep Dive
 4. Transformers & Self-Attention
 5. Parameters
 6. Large Language Models (LLMs)
 7. Training vs Inference
 8. Model Optimization Techniques
 9. Transfer Learning
 10. Fine-tuning
 11. Retrieval-Augmented Generation (RAG)
 12. Prompting, Prompt Engineering, Prompt Tuning
 13. Real-World Industry Applications
 14. Hands-On Project & Security Findings
 15. Overall Reflection
-

1. The Foundational Hierarchy

AI (broadest – any intelligent behavior, rule-based OR learning-based)

└─ ML (learns patterns from data instead of fixed rules)

└─ Deep Learning (multi-layer neural networks)

└─ Foundation Models (huge, broadly-trained base models)

└─ GenAI (generates new content, not just predictions/labels)

- **AI** – any system exhibiting intelligent behavior; includes rule-based ("symbolic") systems, not just learning-based ones.
- **ML** – systems that learn patterns from data (Data + Output → Rules), a flip from traditional programming (Data + Rules → Output).
- **GenAI** – models that generate new content (text, images, audio) rather than classify or predict a fixed label.

Types of AI by capability: Narrow AI (ANI – does one task well; this includes ALL current LLMs), General AI (AGI – theoretical), Super AI (ASI – theoretical).

2. Machine Learning Deep Dive

Core pipeline: Data Collection → Preprocessing → Feature Engineering → Model Selection → Training → Evaluation → Tuning → Deployment → Monitoring.

Types by learning style:

Type	Data	Goal	Example
Supervised	Labeled	Predict outcomes	Spam detection, price prediction
Unsupervised	Unlabeled	Find hidden structure	Clustering, anomaly/fraud detection
Semi-supervised	Small labeled + large unlabeled	Cheap learning at scale	Medical imaging
Self-supervised	Unlabeled, auto-generated labels	Rich representation learning	LLM pre-training
Reinforcement Learning	Reward signal, no fixed dataset	Learn optimal actions	Game AI, robotics, RLHF

Key concepts:

- **Overfitting** – model memorizes training data/noise instead of the general pattern (high train accuracy, poor test accuracy).
- **Deep Learning** – ML using neural networks with many layers; automatically learns features from raw data (vs. manual feature engineering).

3. Generative AI Deep Dive

Definition: Models that learn the underlying data distribution and generate new content resembling it, rather than just classifying/predicting.

Architectures by content type:

Architecture	Best For
GANs (Generator vs Discriminator)	Images, deepfakes
VAEs	Image generation, compression
Diffusion Models	Current standard for images/video (Midjourney, Stable Diffusion)
Transformers	Text, code, and increasingly everything – powers all major LLMs

Key sub-concepts: Foundation Models, LLMs, Multimodal Models, Prompting, Fine-tuning, RAG.

4. Transformers & Self-Attention

Introduced in "Attention Is All You Need" (2017). Solved the sequential-processing/forgetting weakness of older RNN/LSTM models by processing an **entire sequence simultaneously**.

- **Self-Attention** – every token is compared against every other token via Query, Key, Value vectors, producing a context-aware representation.
- **Multi-head attention** – multiple attention "heads" learn different types of relationships in parallel (grammar, coreference, topic).
- **Positional Encoding** – injects word-order information, since attention alone doesn't preserve sequence order.
- **Decoder-only architecture** – used by GPT, Claude, and most modern LLMs; generates text autoregressively.

 **Security relevance:** Attention treats all tokens – trusted instructions and untrusted input/documents alike – as just tokens. There's no hard architectural wall separating "commands" from "data." This is the root mechanical cause of prompt injection.

5. Parameters

- **Weights + biases** – the adjustable numerical values learned during training; this is where a model's "knowledge" is stored.
- Set automatically via training (gradient descent + backpropagation), unlike **hyperparameters** (learning rate, batch size, etc.), which are set manually before training.

 **Security relevance:** Model theft = the illegitimate extraction/copying of these learned parameters, valuable because they represent enormous training investment.

6. Large Language Models (LLMs)

Definition: Transformer-based models trained to predict the next token given prior context, repeatedly (autoregressive generation).

Training pipeline:

Pre-training (self-supervised, massive raw text)
→ Supervised Fine-Tuning / SFT (curated instruction examples)
→ RLHF (human preference ranking → aligned, helpful, safer model)

Key concepts:

- **Tokens** — the actual units LLMs process (not full words).
- **Context window** — the model's short-term memory limit for a single conversation.
- **In-context learning** — learning a task from examples given directly in the prompt.
- **Hallucination** — confident but factually wrong output; a consequence of statistical next-token generation, not "lying" or fact-lookup failure.
- **Knowledge cutoff** — the model has no knowledge of events after its training data was collected.

 **Security relevance:** SFT and RLHF datasets are smaller and more "trusted" than pre-training data, making them higher-leverage targets for data poisoning — a malicious example has proportionally more influence.

7. Training vs Inference

	Training	Inference
Parameters	Change (learning)	Fixed (frozen, applying)
Timing	Offline, rare, expensive	Live, constant, per-request
Data	Massive datasets	Single input at a time
Primary attack	Data Poisoning	Prompt Injection, Jailbreaking

Inference mechanics (deep dive):

- **Logits → Softmax → Token selection** — raw scores converted to probabilities, then a token is chosen.

- **Decoding strategies** – Greedy (always top choice), Temperature Sampling (controlled randomness), Top-k, Top-p (Nucleus).
- **Temperature** – low = deterministic/consistent, high = varied/creative. Security-critical systems (e.g., content moderation) should use **Temperature = 0** for reproducibility, auditability, and to avoid random bypass via sampling variance.
- **KV Caching** – avoids recomputing prior tokens' Key/Value vectors, enabling fast, real-time generation.
- **Batching** – serving many concurrent users efficiently on shared GPU hardware.
- **Latency vs Throughput** – a core tradeoff in production serving.

🔒 **System vs user input** are architecturally just one combined token sequence – the model's tendency to prioritize system instructions is *learned* (via RLHF), not *enforced* – the root cause of jailbreaking.

8. Model Optimization Techniques

Technique	What It Does
Pruning	Removes low-impact weights/structures (unstructured vs structured)
Quantization	Reduces numeric precision of parameters (e.g., 32-bit → 8-bit)
Distillation	Trains a smaller "student" model to mimic a larger "teacher" model

🔒 **Security relevance:** Optimization (especially pruning) can unintentionally strip away RLHF-trained safety behavior even when general task accuracy is preserved – accuracy and safety must be evaluated **separately** after optimization.

9. Transfer Learning

Definition: Reusing knowledge learned on one task/domain for a different but related task, instead of training from scratch.

Transfer Learning (broad strategy)

- └ Feature Extraction (freeze base, train new output layer)
- └ Fine-tuning (unfreeze and retrain layers)
- └ Domain Adaptation (same task, different data distribution)

Fine-tuning is one specific *method* of doing transfer learning; pre-training → fine-tuning in LLMs is a direct application of this concept.

🔒 **Security relevance:** Flaws or vulnerabilities in a base/foundation model propagate to every downstream model fine-tuned from it – a supply-chain-style risk.

10. Fine-tuning

Definition: Continuing training of an already pre-trained model on a smaller, task/domain-specific dataset.

Type	Description
Full Fine-tuning	Updates all parameters – powerful, expensive, risk of catastrophic forgetting
PEFT (e.g., LoRA)	Updates a small set of new "adapter" parameters, base model frozen – cheap, modern default

Good for: domain/task specialization, tone/style, SFT, RLHF.

Not good for: live/frequently-changing information (use RAG), quick simple rules (use prompting).

 **Security relevance:** High-leverage data poisoning target; risk of the model memorizing and leaking sensitive fine-tuning data; risk of catastrophic forgetting degrading safety behavior.

11. Retrieval-Augmented Generation (RAG)

Definition: Grounding an LLM's response in externally retrieved information fetched at query time, rather than relying solely on trained-in knowledge.

Pipeline:

Indexing: Documents → Chunks → Embeddings → Vector Database

Retrieval: Query → Embedding → Semantic Search → Top-matching chunks

Augmentation: Retrieved chunks + Query → Combined Prompt

Generation: LLM answers, grounded in retrieved context

- **Embeddings** – numerical vectors capturing meaning, enabling semantic (meaning-based) search rather than keyword matching.

Benefits: always-current information, reduced hallucination, private/proprietary data access without training exposure, cheaper than fine-tuning, source attribution.

Security relevance (major topic):

- **Access control failures** – retrieval may surface content the querying user isn't authorized to see.
- **Indirect prompt injection** – malicious instructions hidden inside retrieved documents, processed as if legitimate.
- **Knowledge base poisoning** – malicious/false documents inserted into the source data.

- **Accidental sensitive data exposure** through careless retrieval.

12. Prompting, Prompt Engineering, Prompt Tuning

	Training Involved?	Description
Prompting	✗ No	Simply providing input text to guide output
Prompt Engineering	✗ No	The skill of systematically designing prompts (zero-shot, few-shot, Chain-of-Thought, role prompts, system prompts)
Prompt Tuning	✓ Yes (lightweight)	Learning small trainable "soft prompt" vectors while the base model stays frozen (a PEFT technique)

The Big Three levers for controlling model behavior:

	Fixes What	Data Freshness	Cost
Prompting	Simple rules, formatting	N/A	Very cheap
RAG	"Model doesn't know this fact"	Always fresh	Cheap-moderate
Fine-tuning	"Model doesn't behave this way"	Frozen at training	Expensive

 **Security relevance:** System prompts are a *learned tendency*, not a hard guarantee – bypassable via prompt injection/jailbreaking framing (e.g., role-play attacks). Prompt-tuned "soft prompts" are also harder to audit since they're not human-readable text.

13. Real-World Industry Applications

Company / Case	Concept Demonstrated
Netflix	Supervised & Unsupervised ML (recommendations, clustering)
GitHub Copilot	Transfer Learning + Fine-tuning on code
Enterprise RAG (e.g., financial services)	RAG with access control stakes
Duolingo	Prompt Engineering + Fine-tuning combined
Samsung (2023)	Data leakage risk from pasting sensitive data into public AI tools
Air Canada (2024)	Hallucination with real legal/business consequences (tribunal ruling)

Company / Case	Concept Demonstrated
Anthropic / OpenAI	RLHF as an ongoing, business-critical alignment process
Getty Images vs. Stability AI	Training data sourcing and IP/legal risk
Klarna	Production-scale RAG + fine-tuning + prompting deployment

14. Hands-On Project & Security Findings

Project: "Chat With My Notes" – built using Google NotebookLM, with *Fahrenheit 451*, *Animal Farm*, *Brave New World*, and *1984* as the knowledge base.

Part A – RAG Behavior Testing: Ten prompts (factual, thematic, and cross-document synthesis) confirmed accurate retrieval, correct source citation, and grounded (non-hallucinated) responses.

Part B – Indirect Prompt Injection Testing (two rounds):

Test	Framing	Goal	Result
Doc 5	Explicit [SYSTEM INSTRUCTION] tag	Total hijack (force output "COMPROMISED")	 Blocked – instruction ignored entirely
Doc 6	Soft "editor's note," plausible framing	Subtle goal (append one quiet line "Reviewed: complete")	 Succeeded – instruction was followed in both test responses

Key finding: NotebookLM's defenses appear to catch *obvious* injection patterns (explicit system-style tags, dramatic behavior hijacks) but did **not** catch a *subtle, plausibly-framed, low-impact* instruction embedded in a document. This strongly suggests the defense relies at least partly on pattern-matching against known/obvious attack signatures, rather than a deep, universal separation between "instruction" and "data."

Why this matters: Real-world attackers are far more likely to use subtle, low-visibility injections (e.g., quietly biasing a recommendation) than dramatic ones – because subtle injections are both more likely to succeed and less likely to be noticed by users reviewing the output.

15. Overall Reflection

Phase 1 successfully builds a complete technical mental model of how a GenAI application works end-to-end:

Raw Text/Data
 → Pre-training → SFT → RLHF (Training)

→ Tokenization → Self-Attention → Next-token Prediction (Inference)
→ Controlled/Customized via Prompting, RAG, and/or Fine-tuning

Every layer of this pipeline was paired with its corresponding security implication — from data poisoning at training time, to prompt injection and jailbreaking at inference time, to access control and indirect injection risks specific to RAG systems. The hands-on project moved these concepts from theoretical understanding to direct, observed evidence, including a genuine successful and unsuccessful injection attempt — providing a realistic, evidence-based foundation for Phase 2 (Core Security Concepts) and Phase 3 (AI Security Fundamentals).

Phase 1 Goal – Achieved: *"Be able to explain how a GenAI application works,"* now grounded in both conceptual depth and direct practical evidence.

Prepared as part of a self-directed AI Security learning roadmap — Phase 1 of 6.